

Copilot Handoff: Refactor Playbooks Into Goal-Based Execution Workspace

We are continuing the `feature/visual-playbooks-refactor` branch.

Core direction change

The current playbook feature is too centered around the Playbook Detail page. We need to separate the concepts more cleanly:

- **Playbooks** are reusable visual execution guides.
- **Company Goals** are active execution workspaces.
- **Goal Account Responses** are optional rep feedback per assigned account.
- **Posts/Displays** are proof that execution happened.

Do not redesign the whole app. Keep this incremental and compatible with existing Firestore docs.

The desired outcome:

```
Admin creates/edits a company goal
→ optionally attaches one playbook to that goal
→ reps open their assigned goal
→ reps see only the accounts assigned to them for that goal
→ reps use the attached playbook as a sales/execution guide
→ reps optionally mark accounts as playable or not playable
→ if playable, reps may forecast cases, ask for help, or add notes
→ reps later create a display and attach the goal
→ completed display rolls up to the goal and indirectly to the playbook
```

Important: **Create Display should not become the place where reps study or choose playbooks.**

Reps use playbooks before creating the display. When they create the display, they should mainly attach the correct goal. Since the goal points to a playbook, the system can infer the playbook connection.

1. Terminology

Use this product language:

- Customer-facing: **Display**, not Post, when easy to change.
- **Playbook** = reusable visual guide.
- **Goal** = active company execution assignment.
- **Playable** = this account can run the play.

- **Not playable** = this account cannot execute; reason required.
- Avoid making “forecast” the main word in the rep UI. Forecasting is optional under “Playable.”

Do not rename all code from Post to Display right now. Keep code types stable unless already being touched.

2. Final data model

2.1 Playbook model

Playbooks continue to live in the existing library/collections structure.

Keep backward compatibility:

```
collectionType?: "collection" | "playbook";
```

A playbook is reusable and should not own active execution state.

Expected playbook fields:

```
export type PlaybookStatus = "draft" | "published" | "archived";

export interface PlaybookPostSnapshot {
  postId: string;
  imageUrl?: string;
  accountName?: string;
  accountNumber?: string;
  brands?: string[];
  brandIds?: string[];
  productType?: string[];
  description?: string;
  totalCaseCount?: number;
  createdAt?: string;
}

export interface LibraryCollection {
  id: string;
  companyId: string;

  collectionType?: "collection" | "playbook";
  playbookStatus?: PlaybookStatus;

  title: string;
```

```

description?: string;

gamePlan?: string;
whenToUse?: string;
executionGoal?: string;
coachNotes?: string;
audience?: string[];

season?: string;
programName?: string;
chain?: string;
chainType?: string;
supplierId?: string | null;
brandIds?: string[];
goalIds?: string[]; // optional denormalized list only, not source of truth

featuredPostIds?: string[];
playbookPostSnapshots?: PlaybookPostSnapshot[];
featuredPostSnapshots?: PlaybookPostSnapshot[];

createdByUid?: string;
createdAt?: string;
updatedAt?: string;
}

```

Notes:

- `goalIds` can be maintained for quick UI display if useful.
- Do not put account-level planning/forecasting inside the playbook doc.
- Do not embed sales rep responses inside playbooks.

2.2 Company Goal model

A goal may optionally attach **one** playbook.

Do not build multiple playbooks per goal right now. The product assumption is that playbooks are narrow enough in scope that a goal does not need multiple playbooks.

Add these optional fields to the company goal type:

```

export interface CompanyGoalWithIdType {
  id: string;
  companyId: string;
}

```

```

goalTitle: string;
goalDescription?: string;

startDate?: string;
endDate?: string;

assignedUserIds?: string[];
assignedAccountNumbers?: string[];

playbookId?: string | null;
playbookTitle?: string | null;
playbookInstructions?: string;
playbookReason?: string;

createdByUid?: string;
createdAt?: string;
updatedAt?: string;
}

```

Notes:

- Keep existing goal fields intact.
- Add these fields wherever the existing Company Goal type is defined.
- If current assignment fields use different names, do not break them. Add compatibility helpers/selectors instead of large rewrites.
- `playbookId` is optional. Some goals may not use playbooks.

2.3 New collection: `goalAccountResponses`

Create a new Firestore collection:

```
goalAccountResponses/{responseId}
```

This collection stores optional feedback from a rep for one account on one goal.

```

export type GoalAccountStatus = "playable" | "not_playable";

export type NotPlayableReason =
  | "low_volume"
  | "no_room"
  | "sales_do_not_support"
  | "manager_refused"
  | "out_of_stock"

```

```

    | "wrong_account_type"
    | "other";

export interface GoalAccountResponse {
  id: string;
  companyId: string;

  goalId: string;
  playbookId?: string | null;

  accountNumber: string;
  accountName?: string;

  userId: string;
  userFirstName?: string;
  userLastName?: string;
  salesRouteNum?: string;

  accountStatus: GoalAccountStatus;

  // Optional planning fields. Do not require these.
  estimatedCases?: number;
  helpNeeded?: string[];
  repNotes?: string;

  // Required only when accountStatus === "not_playable"
  notPlayableReason?: NotPlayableReason;
  notPlayableNote?: string;

  // Completion can be stored here or derived from posts.
  submittedPostId?: string | null;
  completedAt?: string | null;

  createdAt: string;
  updatedAt?: string;
}

```

Validation rule:

- If `accountStatus === "not_playable"`, require `notPlayableReason`.
- If `accountStatus === "playable"`, do not require estimated cases.
- Reps should be encouraged to respond, not forced.

Unique response key should conceptually be:

```
companyId + goalId + accountNumber + userId
```

Use either:

```
goalAccountResponses/{goalId_accountNumber_userId}
```

or use an auto ID with a query/upsert helper. Prefer deterministic ID if it simplifies upserts and avoids duplicates.

Recommended deterministic ID helper:

```
const getGoalAccountResponseId = (
  goalId: string,
  accountNumber: string | number,
  userId: string
) => `${goalId}_${accountNumber}_${userId}`;
```

3. Frontend vs backend responsibilities

Frontend responsibilities

Implement in React/Firebase client:

1. Admin attaches a playbook while creating/editing a company goal.
2. Goal detail/workspace displays the attached playbook.
3. Rep goal view shows only assigned accounts relevant to the current user.
4. Rep can mark assigned accounts:
 5. Playable
 6. Not playable
 7. If playable:
 8. estimated cases optional
 9. help needed optional
 10. notes optional
 11. If not playable:

12. reason required
13. note optional
14. Admin goal view summarizes:
15. assigned accounts
16. playable count
17. not playable count
18. no response count
19. completed display count
20. estimated case total
21. help needed accounts
22. blocker reason counts
23. Playbook detail shows historical goal usage only, not the active planning form.

Backend / Cloud Functions responsibilities

Do not add Cloud Functions for the first pass unless already required by the project structure.

Recommended first pass:

- Use client-side Firestore helpers for `goalAccountResponses`.
- Use existing post submission flow for goal submission.
- Derive completed displays from existing posts with `companyGoalId`.
- If a post has a goal and that goal has a playbook, include playbook metadata on the post payload for easier reporting.

Optional later backend function:

```
onPostCreated/onPostUpdated
→ if post.companyGoalId exists
→ look up goal
→ if goal.playbookId exists, backfill post.playbookId/playbookTitle
→ upsert goalAccountResponses status/completedAt/submittedPostId
```

Do not build this Cloud Function now unless it is clearly easy and safe. For now, prefer frontend helpers and existing submission logic.

Security rules will need to allow:

- admins to read/write playbooks and goal playbook fields
- normal users to read published playbooks for their company
- users to create/update their own `goalAccountResponses`
- admins/supervisors to read responses for their company

- users should not edit other users' responses unless admin/supervisor
-

4. Files to update

Types

src/types/library.ts

Add or confirm:

- PlaybookStatus
- PlaybookPostSnapshot
- GoalAccountStatus
- NotPlayableReason
- GoalAccountResponse
- CreateGoalAccountResponseInput
- GoalAccountResponseSummary

Recommended additions:

```
export interface CreateGoalAccountResponseInput {
  companyId: string;
  goalId: string;
  playbookId?: string | null;

  accountNumber: string;
  accountName?: string;

  userId: string;
  userFirstName?: string;
  userLastName?: string;
  salesRouteNum?: string;

  accountStatus: GoalAccountStatus;

  estimatedCases?: number;
  helpNeeded?: string[];
  repNotes?: string;

  notPlayableReason?: NotPlayableReason;
  notPlayableNote?: string;
}

export interface GoalAccountResponseSummary {
```



```

    assignedAccountsCount: number;
    playableCount: number;
    notPlayableCount: number;
    noResponseCount: number;
    completedCount: number;
    estimatedCasesTotal: number;
    helpNeededCount: number;
    blockerCounts: Partial<Record<NotPlayableReason, number>>;
  }

```

`src/utils/types.ts`

Extend the existing company goal and post types.

For goals, add:

```

playbookId?: string | null;
playbookTitle?: string | null;
playbookInstructions?: string;
playbookReason?: string;

```

For posts/displays, add optional fields:

```

playbookId?: string | null;
playbookTitle?: string | null;

```

If current code already added `playSourcePostId` or `playName`, keep them for backward compatibility, but stop making Create Display depend on a manual playbook dropdown.

New helper file

Create:

`src/components/Goals/goalAccountResponseHelpers.ts`

or if goal helpers already live elsewhere, place it near existing goal utilities.

Functions:

```

getGoalAccountResponseId(goalId, accountNumber, userId)

```

```
sanitizeGoalAccountResponsePayload(payload)

upsertGoalAccountResponse(input)

fetchGoalAccountResponsesForGoal(companyId, goalId)

fetchGoalAccountResponsesForUserGoal(companyId, goalId, userId)

buildGoalAccountResponseSummary(goal, responses, completedPosts)
```

Implementation notes:

- Use `setDoc(docRef, payload, { merge: true })` for upsert.
- Strip `undefined` fields before writing.
- Preserve `createdAt` if possible; always update `updatedAt`.
- Use deterministic doc ID to prevent duplicates.

5. UI refactor by area

5.1 Playbooks Library page

File likely impacted:

```
src/components/Playbooks/PlaybooksPage.tsx
```

Keep:

- list of playbooks
- create playbook modal for admins only
- read access for non-admin users
- playbook cards

Change:

- Remove or stop emphasizing active forecasting from the Playbooks page.
- Do not load all forecast/account response data for every playbook by default.
- Show lightweight metadata:
 - title
 - description
 - status
 - season/program
 - number of examples
 - maybe “Used in X goals” if already available cheaply

Admin-only:

- create playbook
- edit playbook
- archive/draft/publish

Non-admin:

- view only

Performance:

- Do not query all `goalAccountResponses` from the library page.
- If showing “Used in X goals,” query goals by `playbookId` or use denormalized `goalIds`.

5.2 Playbook detail page

File likely impacted:

```
src/components/Playbooks/PlaybookDetailView.tsx  
src/components/Playbooks/playbookDetailView.css
```

Change intent:

The Playbook Detail page should be a reusable guide and historical reporting page.

It should include:

1. Playbook title/status
2. Description
3. Game plan
4. When to use
5. Execution goal
6. Coach notes
7. Featured display examples
8. Related goals / Used in goals
9. Collapsible historical summary by goal

Remove or de-emphasize:

- Active “Run the Play” forecast form from this page.
- Rep account planning inside playbook detail.

Add:

```
Used in Goals
- Goal title
- Start/end dates
- Status if available
- Assigned accounts count
- Playable count
- Not playable count
- Completed displays count
- Link/open goal workspace
```

The playbook detail can display summaries, but the goal remains the source of truth.

5.3 Company Goal create/edit flow

Files unknown; search for existing company goal create/edit components.

Likely areas:

```
src/components/Goals/*
src/components/CompanyGoals/*
src/Slices/companyGoalsSlice.ts
src/thunks/companyGoalsThunk.ts
```

Add an optional “Attach Playbook” section for admins.

Fields:

```
playbookId
playbookTitle
playbookReason
playbookInstructions
```

UI copy:

```
Attach a Playbook
Optional. Use a playbook to give the team visual examples and execution guidance
for this goal.
```

Inputs:

- Select published playbook

- Reason this playbook applies
- Goal-specific instructions

Rules:

- One playbook per goal.
- Playbook attachment is optional.
- Admin can attach a playbook during goal creation.
- Admin can attach/change/remove a playbook later if the goal is still active.

If the current goal creation modal is already complex, keep this section collapsed by default.

5.4 Goal detail/workspace page

This is the most important UI change.

Find or create a goal detail/workspace component.

Recommended name if missing:

```
src/components/Goals/GoalExecutionWorkspace.tsx
src/components/Goals/goalExecutionWorkspace.css
```

This screen should behave differently for admins vs reps.

Rep view

Rep should see:

- goal title
- goal description
- date range
- attached playbook card if present
- button/link: "Open Playbook"
- only accounts assigned to this rep for this goal
- account response controls

Each assigned account card:

```
Account Name
Address / account number

Can this account run the play?
[Playable] [Not Playable]
```

If Playable:

- Estimated cases optional
- Help needed optional multi-select
- Notes optional
- Save

If Not Playable:

- Reason dropdown required
- Note optional
- Save

Help-needed options can be simple:

```
[  
  "Need POS",  
  "Need supplier help",  
  "Need manager approval",  
  "Need product",  
  "Need pricing support",  
  "Need display space",  
  "Other"  
]
```

Not playable reasons:

```
[  
  { value: "low_volume", label: "Low volume" },  
  { value: "no_room", label: "No room" },  
  { value: "sales_do_not_support", label: "Sales do not support" },  
  { value: "manager_refused", label: "Manager refused" },  
  { value: "out_of_stock", label: "Out of stock" },  
  { value: "wrong_account_type", label: "Wrong account type" },  
  { value: "other", label: "Other" }  
]
```

Important:

- Rep response is encouraged, not required.
- Do not block Create Display if no response exists.
- Playable does not require estimated cases.
- Not playable requires a reason.

Admin view

Admin should see:

- goal details
- attached playbook
- assigned account summary
- response summary
- completed displays
- blocker breakdown
- table/list of account responses

Summary cards:

Assigned accounts
Playable
Not playable
No response
Completed displays
Estimated cases
Needs help

Blocker breakdown:

No room
Low volume
Manager refused
Out of stock
Sales do not support
Wrong account type
Other

Admin should be able to review but not necessarily edit every rep response in this first pass.

5.5 Create Display flow

Files impacted:

```
src/components/Create-Post/PickStore.tsx  
src/utils/PostLogic/buildPostPayload.ts  
src/utils/PostLogic/handlePostCreation.ts  
src/utils/types.ts
```

```
src/components/PostCard.tsx
src/components/postCard.css
```

Direction:

- Remove or stop expanding the optional Playbook Play dropdown in `PickStore.tsx`.
- Do not require reps to select a playbook or source display when creating a display.
- Keep account selection.
- Keep company goal selection.
- When a selected company goal has `playbookId`, show a small non-blocking hint:

This goal uses the "[Playbook Title]" playbook.

- Persist playbook metadata onto the post automatically if the selected goal has it:

```
playbookId: selectedGoal.playbookId ?? null
playbookTitle: selectedGoal.playbookTitle ?? null
```

- Continue saving `companyGoalId` and `companyGoalTitle`.
- Keep existing Gallo goal behavior untouched.

Post submission:

- Existing `handlePostCreation.ts` already calls goal update logic when `post.companyGoalId` exists.
- Add optional logic after successful post create:
- if post has `companyGoalId` and `accountNumber`, upsert matching `goalAccountResponses` with:
 - `submittedPostId`
 - `completedAt`
- Do not overwrite `accountStatus` unless no response exists.
- If no response exists, create a minimal response only if safe:
 - `accountStatus: "playable"`
 - `submittedPostId`
 - `completedAt`
- This should be done carefully to avoid creating bad data.

Safer first pass:

- Do not auto-create responses on post submit.
- Instead, admin completed counts can be derived from posts where `companyGoalId === goal.id`.
- This avoids extra write bugs.

Preferred first pass:

Derive completion from posts, do not force response creation during post submission.

5.6 PostCard

File:

```
src/components/PostCard.tsx  
src/components/postCard.css
```

Current branch may show “Run the Play” banner when `playName` exists.

Change direction:

- Show goal/playbook context only if present.
- Avoid making it look like the rep manually selected a play.
- Suggested copy:

```
Built for goal: [Goal Title]  
Guided by playbook: [Playbook Title]
```

Only show playbook line if `post.playbookTitle` exists.

Keep backward compatibility:

- If `playName` exists from current branch, still display it gracefully, but do not require it going forward.

6. Redux/hooks strategy

Do not add a large new Redux slice unless truly needed.

Preferred:

- Use existing company goals slice for goals.
- Use local component state for `goalAccountResponses` inside goal detail/workspace.
- Add helper hooks only if needed:

```
useGoalAccountResponses(goalId)
useGoalExecutionSummary(goal, responses, posts)
```

Avoid:

- Loading all responses globally.
- Loading all playbook response data on the Playbooks library page.
- Snapshot listeners unless the page needs live updates.

Recommended reads:

- Goal workspace:
- read one goal
- read assigned accounts already available in goal/account state
- read responses where `companyId == user.companyId && goalId == goal.id`
- read posts where `companyGoalId == goal.id` if needed for completion summary

Indexes may be needed for:

```
goalAccountResponses: companyId + goalId
goalAccountResponses: companyId + goalId + userId
posts: companyGoalId
companyGoals: companyId + playbookId
```

7. Firestore security rules

Add rules for `goalAccountResponses`.

Intent:

- Authenticated company users can read responses for their company if allowed.
- Reps can create/update only their own response docs.
- Admin/supervisor can read all company responses.
- Admin/supervisor may update if needed, but first pass can restrict editing to owner/admin.
- Validate required fields.
- Validate `notPlayableReason` when `accountStatus == "not_playable"`.

Pseudo-rule intent:

```
allow read:
  if user belongs to same company and is admin/supervisor
  or resource.data.userId == request.auth.uid
```

```
allow create:
  if request.auth.uid == request.resource.data.userId
  and request.resource.data.companyId == user.companyId
  and valid response shape

allow update:
  if request.auth.uid == resource.data.userId
  and companyId is unchanged
  or user is admin/supervisor of same company
```

Also ensure playbook writes are admin-only:

- Admin/super-admin can create/edit playbooks.
- Reps can read published playbooks for their company.
- Existing collection docs without `collectionType` should still behave as collections.

8. Implementation order

Milestone 1: Types and schema compatibility

Files:

```
src/types/library.ts
src/utils/types.ts
```

Tasks:

- Add goal playbook fields to CompanyGoal type.
- Add `GoalAccountResponse` types.
- Add post optional `playbookId/playbookTitle`.
- Keep all existing fields backward compatible.

Test:

- TypeScript build.
- Existing collection/playbook pages still load.

Milestone 2: Goal create/edit can attach one playbook

Files:

Goal create/edit components
company goals thunk/helper files
companyGoalsSlice if needed

Tasks:

- Load published playbooks for current company.
- Add optional attach playbook UI.
- Save `playbookId`, `playbookTitle`, `playbookReason`, `playbookInstructions` on goal.
- Allow editing/removing attached playbook while goal is active.

Test:

- Create goal without playbook.
- Create goal with playbook.
- Edit active goal and attach playbook.
- Existing goals still work.

Milestone 3: Goal execution workspace

Files:

`src/components/Goals/GoalExecutionWorkspace.tsx`
`src/components/Goals/goalExecutionWorkspace.css`
`src/components/Goals/goalAccountResponseHelpers.ts`

Tasks:

- Display attached playbook card.
- Rep sees only assigned accounts relevant to them.
- Add Playable / Not Playable controls.
- If Playable, optional estimated cases/help/notes.
- If Not Playable, require reason.
- Save response with deterministic ID.
- Admin sees summary cards and response table.

Test:

- Rep can save playable without estimated cases.
- Rep can save playable with help needed.
- Rep cannot save not playable without reason.
- Admin sees counts update.
- No response count is calculated correctly.

Milestone 4: Refactor Playbook Detail

Files:

```
src/components/Playbooks/PlaybookDetailView.tsx  
src/components/Playbooks/playbookDetailView.css  
src/components/Playbooks/PlaybooksPage.tsx
```

Tasks:

- Remove active forecast/planning form from playbook detail.
- Keep playbook guide content.
- Add “Used in Goals” section.
- Each used goal can show collapsible summary and link to goal workspace.
- Do not load all response data unless detail section is opened.

Test:

- Playbook still opens and displays examples.
- Admin can still edit playbook.
- Rep can read playbook.
- Used goals display without heavy reads.

Milestone 5: Simplify Create Display playbook behavior

Files:

```
src/components/Create-Post/PickStore.tsx  
src/utils/PostLogic/buildPostPayload.ts  
src/utils/PostLogic/handlePostCreation.ts  
src/components/PostCard.tsx  
src/components/postCard.css
```

Tasks:

- Remove/de-emphasize manual Playbook Play dropdown from Create Display.
- Keep account and company goal selection.
- When selected goal has a playbook, show non-blocking hint:
 - This goal uses the “[playbookTitle]” playbook.
- Persist `playbookId/playbookTitle` from selected goal onto post payload when available.
- Update PostCard banner to show goal/playbook context, not manual “Run the Play” as the primary model.

Test:

- Create display with goal that has no playbook.
 - Create display with goal that has playbook.
 - Post stores goal and playbook metadata.
 - Gallo goal behavior still works.
 - Existing posts without playbook fields render normally.
-

Milestone 6: Security rules and indexes

Tasks:

- Add `goalAccountResponses` rules.
 - Confirm playbook write permissions are admin-only.
 - Add or document required indexes.
 - Test common reads/writes as admin and rep.
-

9. What not to do

Do not:

- Rewrite the entire library system.
 - Move all playbook data into goals.
 - Put account-level feedback inside playbook docs.
 - Require reps to respond before creating displays.
 - Require estimated cases when marking an account playable.
 - Add multiple playbooks per goal yet.
 - Add Cloud Functions unless necessary.
 - Load all goal responses on the Playbooks landing page.
 - Break existing collections where `collectionType` is missing.
-

10. Desired end state

After this refactor:

Playbooks are clean reusable visual guides.

Goals are the active place where admins assign work and review field reality.

Reps use the playbook as a sales tool before building the display.

Reps can quickly tell the company whether each assigned account is playable or not playable.

Admins can see why a goal is succeeding or failing:

- completed displays
- playable accounts
- not playable accounts
- no response accounts
- help needed
- blocker reasons

Create Display stays simple:

- select account
- select goal
- submit display
- system connects the display back to the goal and playbook

This should be built incrementally with small, testable changes.